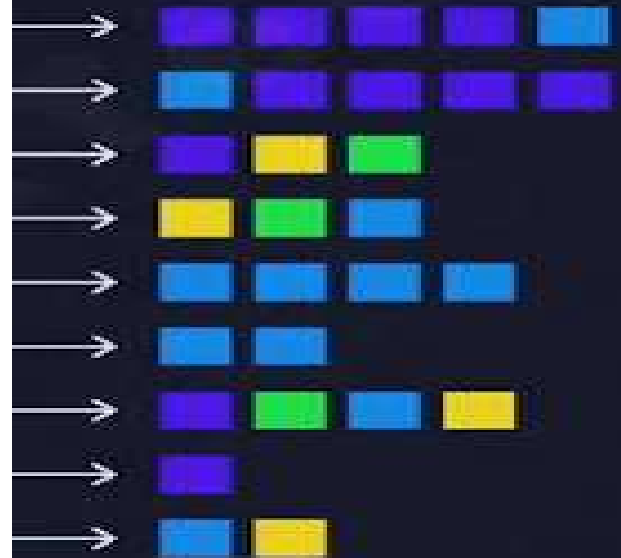




JavaScript-01

Smarta Array-metoder

ARRAY SHEET



18 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Smarta Array-metoder

I JavaScript är **arrayer** och **listor** tekniskt sett två olika saker, men i vardagligt kodsnack använder programmerare ofta orden för att beskriva samma sak.

1. Det korta svaret

Array är den faktiska, officiella datatypen i JavaScript (t.ex. `const arr = []`). Lista är mer av ett allmänt begrepp. Det finns ingen inbyggd datatyp i JavaScript som heter "Lista". När en JavaScript-utvecklare säger "lista", menar de nästan alltid en helt vanlig array.

2. Den tekniska skillnaden

Även om man använder orden på samma sätt i vardagen, skiljer de sig åt om man tittar under huven rent datatekniskt:

Funktion	JavaScript-array	Datateknisk "lista"
Vad det är	Ett inbyggt objekt i JavaScript som används för att spara en sorterad samling av saker.	Ett teoretiskt begrepp för en sorterad samling av element.
Minneshantering	Dynamisk och sköts helt automatiskt av JavaScript-motorn.	Elementen är oftast länkade till varandra med "pekare" som ligger utspridda i minnet.
Prestanda	Att hämta något med ett index (t.ex. <code>arr[0]</code>) går blixtnabbt.	Att hämta ett element kräver oftast att datorn börjar från början och räknar sig framåt.

Bild 1 Skillnad mellan JavaScript array och lista.

3. När "Lista" betyder något annat i JavaScript

Det finns ett par sällsynta undantag inom webbutveckling där ordet "lista" dyker upp och faktiskt inte betyder en vanlig array:

NodeList

När du hämtar HTML-element med JavaScript (t.ex. med `document.querySelectorAll('p')`) får du tillbaka något som kallas för en `NodeList`. Den ser ut och fungerar nästan som en vanlig array, men den saknar många av de vanliga inbyggda funktionerna (som `.map()` eller `.reduce()`). Man brukar ofta omvandla den med `Array.from(minNodeList)` för att kunna jobba med den som en vanlig array.

Egna datastrukturer

Om du bygger avancerade algoritmer kan du programmera en egen Länkad lista (`Linked List`) från grunden med hjälp av JavaScript-klasser.

Bild 2 `NodeList` och egna datastrukturer

4. Sammanfattning

När du skriver vanlig JavaScript-kod kommer du alltid att använda klamrar, eller hakparenteser `[ ]`. Men det går alldeles utmärkt att kalla det för en "lista" när du pratar med andra utvecklare – alla kommer att förstå precis vad du menar.

Hittills har vi mest använt vanliga for-loopar när vi velat gå igenom listor (arrays). Det fungerar, men det blir snabbt mycket kod att skriva (skapa en räknare, hålla koll på `i < array.length`, plussa på `i++`). Moderna JavaScript-utvecklare använder nästan aldrig for-loopar. I stället använder man inbyggda **array-metoder**. De gör exakt samma sak, men med mycket mindre kod, och koden blir betydligt lättare att läsa. Vi går igenom de vanligaste och de du har mest nytta av i de kommande momenten.

.forEach() – Den enkla loopen

Om du bara vill loopa igenom en lista och göra något med varje sak (t.ex. skriva ut det i konsolen), då är det `.forEach()` du ska välja. I den skickar man in en funktion (ofta en arrow function) direkt i `.forEach()`. JavaScript kommer sedan att köra den funktionen en gång för varje sak i listan.

```
const frukter = ['äpple', 'banan', 'apelsin']

// För varje frukt i listan, kör console.log()
frukter.forEach((frukt) => {
  console.log(`Jag gillar ${frukt}!`)
});

// Utskrift i konsolen
Jag gillar äpple!
Jag gillar banan!
Jag gillar apelsin!
```

Bild 3 En enkel forEach-loop av en frukt-array.

På själva console.log-raderna är det smidigast att använda en så kallad 'template literal'. Alternativet är att klippa ihop en sträng som i Java till exempel med plustecken mellan delarna.

.map() – Gör om en lista till en ny lista

Det här är en av de viktigaste metoderna (och den absolut viktigaste för React). .map() används när du har en lista, vill ändra på alla saker i listan, och få tillbaka en **helt ny** lista med de ändrade sakerna. Den ursprungliga listan rörs inte.

Inkommande array	→	[ ,  , ]
Metoden .map()	→	[Skala frukten]
		(skalad) (skalad)
Ny array ut	→	[ ,  , ]

Bild 4 En förklaring hur array-metoden .map() fungerar.

På bild 4 ser man principen för hur .map() fungerar. Man loopar igenom hela arrayen, element för element. För varje element gör man någonting, i det här fallet skalar en frukt, om den inte redan är skalad eller inte behöver skalas. Som resultat får man tillbaka en ny array med det nya innehållet. Den gamla arrayen som man använde från början finns fortfarande kvar, oförändrad, som före man körde metoden.

```
let priserDollar = [10, 20, 50]

let priserSEK = priserDollar.map((pris) => {
  return pris * 10
})

console.log(priserSEK)
console.log(priserDollar)

// Utskrift i konsolen
[ 100, 200, 500 ]
[ 10, 20, 50 ]
```

Bild 5 Ett kodexempel med array-metoden .map().

I kodexemplet på bild 5 ser man att ursprungs-arrayen är kvar precis som innan, men den nya arrayen med svenska priser har man fått som resultat av körningen.

.filter() – Sortera bort det du inte vill ha

Den här metoden går igenom alla element i din array och behåller bara de du vill ha kvar. De stannar på filtret. Man jämför med ett villkor, de element som uppfyller villkoret är de som stannar kvar och de andra elementen filtreras bort. Även i det här fallet får man tillbaka en helt ny array och ursprungs-arrayen är kvar helt oförändrad.

```
const aldrar = [10, 17, 24, 38, 45, 52, 69, 85]

let vuxna = aldrar.filter((alder) => {
  return alder >= 18
})

console.log("vuxna => ", vuxna)
console.log("aldrar => ", aldrar)

// Utskrift i konsolen
vuxna => [ 24, 38, 45, 52, 69, 85 ]
aldrar => [ 10, 17, 24, 38, 45, 52, 69, 85 ]
```

Bild 6 Ett kodexempel med array-metoden .filter().

.find() – Hitta en specifik sak

Metoden .filter() hittar **alla** element som uppfyller ett villkor och skickar tillbaka en lista med alla dem elementen i. Metoden .find() däremot hittar bara **det första** elementet som uppfyller villkoret, och skickar bara tillbaka det elementet, inte en lista. Så fort metoden har hittat ett element så avbryter den och skickar tillbaka resultatet. Om den inte skulle hitta ett enda element som stämmer överens med ditt villkor, så skickar den tillbaka **undefined**.

```
const personer = [
  { id: 1, namn: "Anna" },
  { id: 2, namn: "Benny" },
  { id: 3, namn: "Cecilia" }
]

// Hitta personen som har id 2
const personFinns = personer.find((user) => {
  return user.id === 2
})

// Hitta personen som har id 7
const personSaknas = personer.find((user) => {
  return user.id === 7
})

console.log("--> personFinns", personFinns)
console.log("--> personSaknas", personSaknas)

// Utskrift i konsolen
personFinns => { id: 2, namn: 'Benny' }
personSaknas => undefined
```

Bild 7 Ett kodexempel med array-metoden .find().

.reduce() – Slå ihop hela listan till en sak

Metoden .reduce() är den svåraste array-metoden att förstå sig på, men den är användbar. Om du vill räkna ihop totalsumman för alla varor du har i din kundvagn, när du handlar på nätet, är .reduce() bra. Den tar två argument i sin funktion.

- **Ackumulatorn (totalt)** - variabeln som sparar det ihopräknade värdet under resans gång.
- **Det nuvarande elementet** – elementet man är på just nu i loopen.

Dessutom skickar vi med ett startvärde (oftast 0) i slutet av metoden. I bild 6 nedan ser du principen för hur .reduce() fungerar.

```
Loopen startar (Startvärde: 0)
Steg 1:      0 (totalt) + 150 kr (tröja)          = 150 kr
Steg 2: 150 (totalt) + 400 kr (skor)              = 550 kr
Steg 3: 550 (totalt) + 50 kr (strumpor) = 600 kr
Slutresultat ut: 600 kr
```

Bild 8 Principen för hur metoden .find() fungerar.

Säg att vi har den här kundvagnen nu (tröja, skor, strumpor, skjorta), och vill räkna ut totalvärdet för den. Med bara priset i arrayen skulle det se ut som följande kodexempel.

```
// A shopping cart array without using square brackets
const kundvagn = new Array(100, 150, 50, 300);

// Räkna ut totalsumman. Vi sätter startvärdet till 0 längst ner.
const totalsumma = kundvagn.reduce((totalt, pris) => {
  return totalt + pris;
}, 0);

console.log(totalsumma); // Utskrift: 600
```

Bild 9 Metoden .find() rent kodmässigt.

Tittar man närmare på metoden reduce() så ser den ut som nedan i detalj.

```
const callbackFunction = function(totalt, pris) { return totalt + pris }
const initialValue = 0

reduce(callbackFunction, initialValue)
```

Bild 10 Metoden .reduce() i detalj.

Enligt bild 10 så tar den vanligaste formen av `reduce()`, två argument, en funktion (= `callbackFunction`) och ett startvärde (= `initialValue`). Så i princip kan man skriva den som ovan så ser den inte lika skräckinjagande ut. Så för varje element i den arrayen man har så anropas först funktionen med argument totalt som är startvärdet, som första gången är 0, plus priset för varan. Andra varvet är det 0 plus priset för första varan plus priset för andra varan. Så fortsätter det tills man har stegat igenom alla element i arrayen. Vi ser på ett sista kodexempel innan vi lämnar array-metoderna.

```
const initialValue = 0
const kundvagn = [
  { vara: "byxor", pris: 125 },
  { vara: "tröja", pris: 200 },
  { vara: "skjorta", pris: 300 },
]

const callbackFunction = function (totalt, el) {
  console.log("-----")
  console.log("--> totalt", totalt, el.vara, el.pris)
  return totalt + el.pris
}

let totaltPris = kundvagn.reduce(callbackFunction, initialValue)

console.log("-----")
console.log("--> totaltPris", totaltPris)

// Utskrift i konsolen
-----
--> totalt 0 byxor 125
-----
--> totalt 125 tröja 200
-----
--> totalt 325 skjorta 300
-----
--> totaltPris 625
```

Bild 11 Metoden `.reduce()` som kodexempel

Som förväntat anropas funktionen tre gånger med först värdet 0 på totalt och byxor, andra gången 125 på totalt och tröja och sista gången 325 totalt och skjorta. Från funktionen returneras värdet på total plus priset för aktuell vara.